

```
import nltk
```

```
from nltk.tokenize import word_tokenize
```

```
from nltk.corpus import stopwords
```

```
from nltk.stem import PorterStemmer, WordNetLemmatizer
```

```
from nltk import pos_tag
```

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
nltk.download('punkt')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt.zip.
True
```

```
nltk.download('stopwords')
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Unzipping corpora/stopwords.zip.
True
```

```
nltk.download('averaged_perceptron_tagger')
```

```
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data] /root/nltk_data...
[nltk_data] Unzipping taggers/averaged_perceptron_tagger.zip.
True
```

```
nltk.download('wordnet')
```

```
[nltk_data] Downloading package wordnet to /root/nltk_data...
True
```

```
text = """Tokenization is the first step in text analytics.
The process of breaking down a text paragraph into smaller chunks
such as words or sentences is called Tokenization."""
```

```
from nltk.tokenize import sent_tokenize
nltk.download('punkt_tab')
tokenized_text=sent_tokenize(text)
print(tokenized_text)
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt_tab.zip.
['Tokenization is the first step in text analytics.', 'The process of breaking down a text paragraph into smaller chunks\nsu
```

```
from nltk.tokenize import word_tokenize
tokenized_word=word_tokenize(text)
print(tokenized_word)
```

```
['Tokenization', 'is', 'the', 'first', 'step', 'in', 'text', 'analytics', '.', 'The', 'process', 'of', 'breaking', 'down', ']
```

```
from nltk.corpus import stopwords
stop_words=set(stopwords.words("english"))
print(stop_words)
```

```
{'while', 'myself', 'off', 'into', 'it', 'ourselves', 'against', 'their', 'couldn't', 'did', 'below', 'was', 'been', 'does',
```

```
import re
text= "How to remove stop words with NLTK library in Python?"
text= re.sub('[^a-zA-Z]', ' ',text)
tokens = word_tokenize(text.lower())
```

```

filtered_text=[]
for w in tokens:
    if w not in stop_words:
        filtered_text.append(w)
print("Tokenized Sentence:",tokens)

```

Tokenized Sentence: ['how', 'to', 'remove', 'stop', 'words', 'with', 'nltk', 'library', 'in', 'python']

```
print("Filterd Sentence:",filtered_text)
```

Filterd Sentence: ['remove', 'stop', 'words', 'nltk', 'library', 'python']

```

from nltk.stem import PorterStemmer

ps = PorterStemmer()
words = ["wait", "waiting", "waited", "waits"]

for w in words:
    rootWords=ps.stem(w)
    print(rootWords)

```

wait
wait
wait
wait

```

from nltk.stem import WordNetLemmatizer

lemmatizer = WordNetLemmatizer()
text = "studies studying cries cry"

tokens = word_tokenize(text)

for w in tokens:
    print("Lemma for", w, "is", lemmatizer.lemmatize(w))

```

Lemma for studies is study
Lemma for studying is studying
Lemma for cries is cry
Lemma for cry is cry

```

import pandas as pd
import math

```

```

documentA = 'Jupiter is the largest Planet'.lower()
documentB = 'Mars is the fourth planet from the Sun'.lower()

```

```

bagA = documentA.split()
bagB = documentB.split()

```

```
uniqueWords = set(bagA).union(set(bagB))
```

```

numA = dict.fromkeys(uniqueWords, 0)
for word in bagA:
    numA[word] += 1

numB = dict.fromkeys(uniqueWords, 0)
for word in bagB:
    numB[word] += 1

```

```

def computeTF(wordDict, bag):
    tf = {}
    total = len(bag)
    for word, count in wordDict.items():
        tf[word] = count / total
    return tf

tfFirst = computeTF(numA, bagA)
tfSecond = computeTF(numB, bagB)

```

```

def computeIDF(docs):
    N = len(docs)
    idf = dict.fromkeys(docs[0].keys(), 0)

    for doc in docs:
        for word, val in doc.items():
            if val > 0:

```

```

        idf[word] += 1

    for word in idf:
        if idf[word] == 0:
            idf[word] = 0
        else:
            idf[word] = math.log(N / idf[word])

    return idf

ids = computeIDF([numA, numB])

```

```

def computeTFIDF(tf, ids):
    tfidf = {}
    for word, val in tf.items():
        tfidf[word] = val * ids[word]
    return tfidf

tfidfFirst = computeTFIDF(tfFirst, ids)
tfidfSecond = computeTFIDF(tfSecond, ids)

```

```

df = pd.DataFrame([tfidfFirst, tfidfSecond]).fillna(0)
print("\nTF-IDF Table:")
print(df)

```

```

TF-IDF Table:
  the      mars  jupiter  largest  planet      from      sun  fourth \
0  0.0  0.000000  0.138629  0.138629    0.0  0.000000  0.000000  0.000000
1  0.0  0.086643  0.000000  0.000000    0.0  0.086643  0.086643  0.086643

    is
0  0.0
1  0.0

```